Estimaciones_Agricolas

February 23, 2026

1 ———————————————————————————————————————

2 Análisis Integral del Dataset de Estimaciones Agrícolas en Argentina

3 ———————————————————————————————————————

Este proyecto tiene como objetivo realizar un análisis exhaustivo del dataset **"estim-agricolas"**, que contiene información sobre la producción agrícola en Argentina.

El análisis se divide en **cuatro etapas principales**, cada una diseñada para obtener información, limpiar el dataset, generar nuevas métricas y preparar los datos para visualización avanzada.

3.1 1. Análisis Exploratorio de Datos (EDA)

Esta etapa inicial incluye:

- Revisión de estructura del dataset.
- Identificación de tipos de datos.
- Detección de valores nulos.
- Análisis estadístico descriptivo.
- Detección de inconsistencias o valores atípicos.
- Evaluación de necesidad de limpieza previa.

El objetivo del EDA es obtener una comprensión profunda del dataset antes de aplicar transformaciones o modelos.

3.2 2. Ciencia de Datos

Incluye:

- Ingeniería de características (nuevas métricas).
- Detección de correlaciones entre variables.
- Identificación de patrones y relaciones relevantes.
- Posible modelado predictivo:
  - productividad,
  - rendimientos,

  - variabilidad entre ciclos agrícolas.

Estas herramientas permiten generar insights avanzados para análisis técnico y aplicaciones profesionales.

3.3 3. Análisis Espacial

Orientado a:

- Vinculación del dataset con herramientas de visualización territorial como **Power BI**.
- Uso de herramientas GIS como **QGIS** para análisis espacial.
- Mapas temáticos de:
  - cultivos,
  - rendimiento,
  - producción,
  - superficies sembradas y cosechadas.

Este análisis agrega una dimensión geográfica crítica para decisiones en agricultura y planificación territorial.

3.4 4. Aplicaciones Prácticas

Esta etapa transforma los resultados en herramientas útiles:

- Desarrollo de dashboards interactivos en **Power BI**.
- Apoyo a consultoría técnica, planificación territorial y gestión agrícola.
- Comunicación visual para tomadores de decisión.

Incluye la construcción de un **modelo estrella** para el consumo eficiente de datos en Power BI.

3.5 Diccionario del Dataset

El dataset incluye las siguientes columnas:

Columna	Tipo de Dato	Descripción	Unidad de Medida
provincia_id	int	Código de provincia	-
provincia_nombre	str	Nombre de la provincia	-
departamento_id	int	Código de departamento	-
departamento_nombre	str	Nombre del departamento	-
id_cultivo	int	Código del cultivo	-
cultivo	str	Nombre del cultivo	-
id_ciclo	int	Código de la campaña agrícola	-
ciclo	str/int	Año de la campaña	-
sup_sembrada	float	Superficie sembrada	Hectáreas
sup_cosechada	float	Superficie cosechada	Hectáreas
producción	float	Producción total	Toneladas

Columna	Tipo de Dato	Descripción	Unidad de Medida
rendimiento	float	Rendimiento por hectárea	Kilogramos por hectárea

---

Este documento constituye la base conceptual para iniciar el análisis integral del dataset y realizar su posterior implementación en Python, Power BI y herramientas GIS.

[1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
import seaborn as sns
import plotly.express as px
import os
from pathlib import Path
import warnings
warnings.filterwarnings("ignore")

# Opcional: ajustar display
pd.set_option("display.max_columns", None)
```

[2]:
```
# Ajustar display opcional
pd.set_option("display.max_columns", None)

# Función para importar el dataset
def importar_dataset(ruta):
    return pd.read_csv(ruta, sep=',', encoding='utf-8')  # Agregamos separador␣
 ↪y codificación

# Ruta del archivo (sin espacio antes del paréntesis)
ruta_archivo = r"C:
 ↪\Users\victo\OneDrive\Data_Science\Repos_1\estimaciones-agricolas-2023-10␣
 ↪(5).csv"

# Importar el dataset
df = importar_dataset(ruta_archivo)
```

[3]:
```
# Inspeccion Inicial del DataFrame

print("Primeras filas del dataset:")
df.head()
```

```
Primeras filas del dataset:
```

[3]:
```
   provincia_id provincia_nombre  departamento_id     departamento_nombre  \
0             6     Buenos Aires              854              25 de Mayo
1             6     Buenos Aires               14  Adolfo Gonzales Chaves
```

```
2             6     Buenos Aires                21                  Alberti
3             6     Buenos Aires                28          Almirante Brown
4             6     Buenos Aires                77                Arrecifes

   id_cultivo cultivo  id_ciclo      ciclo  sup_sembrada sup_cosechada  \
0         1.0     Ajo         1  1969/1970             3             3
1         1.0     Ajo         1  1969/1970            15            15
2         1.0     Ajo         1  1969/1970             5             5
3         1.0     Ajo         1  1969/1970             2             2
4         1.0     Ajo         1  1969/1970             6             6

  produccion rendimiento
0         10        3333
1         82        5467
2          7        1400
3          8        4000
4         15        2500
```

[4]:
```
df.shape
```

[4]:
```
(153889, 12)
```

[5]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 153889 entries, 0 to 153888
Data columns (total 12 columns):
 #   Column               Non-Null Count   Dtype
---  ------               --------------   -----
 0   provincia_id         153889 non-null  int64
 1   provincia_nombre     153889 non-null  object
 2   departamento_id      153889 non-null  int64
 3   departamento_nombre  153889 non-null  object
 4   id_cultivo           152820 non-null  float64
 5   cultivo              153889 non-null  object
 6   id_ciclo             153889 non-null  int64
 7   ciclo                153889 non-null  object
 8   sup_sembrada         153889 non-null  int64
 9   sup_cosechada        153889 non-null  object
 10  produccion           153889 non-null  object
 11  rendimiento          153889 non-null  object
dtypes: float64(1), int64(4), object(7)
memory usage: 14.1+ MB
```

[6]:
```
df.describe()
```

[6]:
```
         provincia_id  departamento_id     id_cultivo       id_ciclo  \
count  153889.000000    153889.000000  152820.000000  153889.000000
```

```
mean       34.003899       192.228736      19.627810      24.132388
std        29.413859       229.194104      10.233854      16.130805
min         6.000000         0.000000       1.000000       1.000000
25%         6.000000        49.000000      11.000000      10.000000
50%        22.000000        98.000000      22.000000      21.000000
75%        54.000000       196.000000      28.000000      39.000000
max        94.000000       882.000000      35.000000      53.000000

       sup_sembrada
count  1.538890e+05
mean   1.228560e+04
std    3.585014e+04
min    0.000000e+00
25%    1.800000e+02
50%    1.439000e+03
75%    8.500000e+03
max    1.096100e+06
```

[7]:
```
df.isnull().sum()
```

[7]:
```
provincia_id              0
provincia_nombre          0
departamento_id           0
departamento_nombre       0
id_cultivo             1069
cultivo                   0
id_ciclo                  0
ciclo                     0
sup_sembrada              0
sup_cosechada             0
produccion                0
rendimiento               0
dtype: int64
```

[8]:
```
import pandas as pd

# Supongamos que tu DataFrame se llama df

# Primero limpiamos y convertimos las columnas que están como 'object' a␣
↪numéricas
cols_to_int = ["sup_cosechada", "produccion", "rendimiento"]

for col in cols_to_int:
    df[col] = pd.to_numeric(df[col], errors="coerce").fillna(0).astype("int64")

# Convertimos id_cultivo (float64 con nulos) a entero
df["id_cultivo"] = df["id_cultivo"].fillna(0).astype("int64")
```

```python
# Reducimos tamaño de columnas clave a int32
cols_to_int32 = ["provincia_id", "departamento_id", "id_ciclo"]
df[cols_to_int32] = df[cols_to_int32].astype("int32")

# Verificamos cambios
print(df.dtypes)
```

```
provincia_id            int32
provincia_nombre       object
departamento_id         int32
departamento_nombre    object
id_cultivo              int64
cultivo                object
id_ciclo                int32
ciclo                  object
sup_sembrada            int64
sup_cosechada           int64
produccion              int64
rendimiento             int64
dtype: object
```

[9]:
```python
# Crear columna periodo_fecha (tipo datetime)
df["periodo_fecha"] = pd.to_datetime(df["ciclo"].str.split("/").str[0],␣
 ↪format="%Y")

# Crear columna periodo_anio (tipo entero)
df["periodo_anio"] = df["ciclo"].str.split("/").str[0].astype("int32")

# Eliminar la columna anterior 'periodo'
if "periodo" in df.columns:
    df = df.drop(columns=["periodo"])

# Verificar cambios
print(df[["ciclo", "periodo_fecha", "periodo_anio"]].head())
print("\nTipos de datos:\n", df.dtypes)
```

```
       ciclo periodo_fecha  periodo_anio
0  1969/1970    1969-01-01          1969
1  1969/1970    1969-01-01          1969
2  1969/1970    1969-01-01          1969
3  1969/1970    1969-01-01          1969
4  1969/1970    1969-01-01          1969

Tipos de datos:
 provincia_id                    int32
provincia_nombre               object
departamento_id                 int32
```

```
departamento_nombre            object
id_cultivo                      int64
cultivo                        object
id_ciclo                        int32
ciclo                          object
sup_sembrada                    int64
sup_cosechada                   int64
produccion                      int64
rendimiento                     int64
periodo_fecha          datetime64[ns]
periodo_anio                    int32
dtype: object
```

[10]:
```
df.shape
```

[10]: (153889, 14)

[11]:
```
import pandas as pd

# Supongamos que tu DataFrame se llama df

# Lista de columnas a analizar
cols = ["sup_sembrada", "sup_cosechada", "produccion", "rendimiento"]

# Contar cuántos registros son exactamente 0 en cada columna
conteo_ceros = {}
for col in cols:
    # Convertimos a numérico por si alguna columna está como 'object'
    df[col] = pd.to_numeric(df[col], errors="coerce").fillna(0)
    conteo_ceros[col] = (df[col] == 0).sum()

# Mostrar resultados
for col, count in conteo_ceros.items():
    print(f"Columna '{col}': {count} valores iguales a 0")
```

```
Columna 'sup_sembrada': 3225 valores iguales a 0
Columna 'sup_cosechada': 16327 valores iguales a 0
Columna 'produccion': 13161 valores iguales a 0
Columna 'rendimiento': 16362 valores iguales a 0
```

[12]:
```
import pandas as pd

# Supongamos que tu DataFrame se llama df

# Aseguramos que las columnas sean numéricas
cols = ["sup_sembrada", "sup_cosechada", "produccion", "rendimiento"]
for col in cols:
    df[col] = pd.to_numeric(df[col], errors="coerce").fillna(0).astype("int64")
```

```
# Filtrar registros inconsistentes:
# sup_sembrada = 0 pero alguna de las otras > 0
inconsistencias = df[
    (df["sup_sembrada"] == 0) &
    (
        (df["sup_cosechada"] > 0) |
        (df["produccion"] > 0) |
        (df["rendimiento"] > 0)
    )
]

# Mostrar cuántos registros inconsistentes hay
print(f"Total de inconsistencias: {len(inconsistencias)}")

# Opcional: ver algunos ejemplos
print(inconsistencias.head())
```

```
Total de inconsistencias: 3202
     provincia_id provincia_nombre  departamento_id departamento_nombre  \
785            34          Formosa               14             Formosa
786            34          Formosa               21              Laishi
787            34          Formosa               35              Patino
788            34          Formosa               42             Pilagas
789            34          Formosa               49           Pilcomayo

     id_cultivo cultivo  id_ciclo      ciclo  sup_sembrada  sup_cosechada  \
785           6  Banana         1  1969/1970             0              0
786           6  Banana         1  1969/1970             0              0
787           6  Banana         1  1969/1970             0              0
788           6  Banana         1  1969/1970             0              0
789           6  Banana         1  1969/1970             0              0

     produccion  rendimiento periodo_fecha  periodo_anio
785        2400            0    1969-01-01          1969
786        3400            0    1969-01-01          1969
787        1500            0    1969-01-01          1969
788       39000            0    1969-01-01          1969
789      153000            0    1969-01-01          1969
```

[13]:
```
import pandas as pd

# Supongamos que tu DataFrame se llama df

# Aseguramos que las columnas sean numéricas
cols = ["sup_sembrada", "sup_cosechada", "produccion", "rendimiento"]
for col in cols:
```

```python
    df[col] = pd.to_numeric(df[col], errors="coerce").fillna(0).astype("int64")

# Detectar inconsistencias:
# sup_sembrada = 0 pero alguna de las otras > 0
inconsistencias = df[
    (df["sup_sembrada"] == 0) &
    (
        (df["sup_cosechada"] > 0) |
        (df["produccion"] > 0) |
        (df["rendimiento"] > 0)
    )
]

print(f"Total de inconsistencias detectadas: {len(inconsistencias)}")

# Eliminar inconsistencias del DataFrame
df_limpio = df.drop(inconsistencias.index)

print(f"Dataset original: {len(df)} registros")
print(f"Dataset limpio: {len(df_limpio)} registros")
```

```
Total de inconsistencias detectadas: 3202
Dataset original: 153889 registros
Dataset limpio: 150687 registros
```

[14]:
```python
df.isnull().sum()
```

[14]:
```
provincia_id           0
provincia_nombre       0
departamento_id        0
departamento_nombre    0
id_cultivo             0
cultivo                0
id_ciclo               0
ciclo                  0
sup_sembrada           0
sup_cosechada          0
produccion             0
rendimiento            0
periodo_fecha          0
periodo_anio           0
dtype: int64
```

[15]:
```python
df.shape
```

[15]: (153889, 14)

[16]:
```python
import pandas as pd

# Supongamos que ya tienes df limpio
# Verificar duplicados y nulos antes de guardar
print("Duplicados:", df.duplicated().sum())
print("Nulos por columna:\n", df.isnull().sum())

# Guardar dataset limpio en nuevo archivo CSV
df.to_csv(r"C:\Users\victo\OneDrive\Data_Science\Repos_1\estim-agric-limpio.
 ↪csv", index=False)

print("Dataset limpio guardado correctamente en 'estim-agric-limpio.csv'")
```

```
Duplicados: 0
Nulos por columna:
 provincia_id           0
provincia_nombre       0
departamento_id        0
departamento_nombre    0
id_cultivo             0
cultivo                0
id_ciclo               0
ciclo                  0
sup_sembrada           0
sup_cosechada          0
produccion             0
rendimiento            0
periodo_fecha          0
periodo_anio           0
dtype: int64
Dataset limpio guardado correctamente en 'estim-agric-limpio.csv'
```

[17]:
```python
# Después de limpiar inconsistencias
df_limpio = df.drop(inconsistencias.index)

# Guardar correctamente SIN índice
df_limpio.to_csv(
    r"C:\Users\victo\OneDrive\Data_Science\Repos_1\estim-agric-limpio.csv",
    index=False
)

# Volver a cargar para verificar
df = pd.read_csv(r"C:
 ↪\Users\victo\OneDrive\Data_Science\Repos_1\estim-agric-limpio.csv")

print("Shape:", df.shape)  # Esperado: (150687, 12)
```

```
Shape: (150687, 14)
```

[18]:
```
# Guardar dataset limpio correctamente
df_limpio.to_csv(
    r"C:\Users\victo\OneDrive\Data_Science\Repos_1\estim-agric-limpio.csv",
    index=False
)

# Volver a cargar para verificar
df = pd.read_csv(r"C:
 ↪\Users\victo\OneDrive\Data_Science\Repos_1\estim-agric-limpio.csv")

print("Shape:", df.shape)  # Esperado: (150687, 12)
```

```
Shape: (150687, 14)
```

[19]:
```
df.shape
```

[19]:
```
(150687, 14)
```

[20]:
```
df.dtypes
```

[20]:
```
provincia_id            int64
provincia_nombre       object
departamento_id         int64
departamento_nombre    object
id_cultivo              int64
cultivo                object
id_ciclo                int64
ciclo                  object
sup_sembrada            int64
sup_cosechada           int64
produccion              int64
rendimiento             int64
periodo_fecha          object
periodo_anio            int64
dtype: object
```

[21]:
```
# Sobrescribir el mismo archivo con las correcciones
df.to_csv(r"C:\Users\victo\OneDrive\Data_Science\Repos_1\estim-agric-limpio.
 ↪csv", index=False)
```

[22]:
```
import pandas as pd

# Leer siempre el archivo corregido
df = pd.read_csv(r"C:
 ↪\Users\victo\OneDrive\Data_Science\Repos_1\estim-agric-limpio.csv")
```

[23]:
```
# Columnas de interés
cols = [
```

```
    "provincia_id", "provincia_nombre",
    "departamento_id", "departamento_nombre",
    "id_cultivo", "cultivo",
    "ciclo"
]

# Contar valores únicos en cada columna
conteo_unicos = df[cols].nunique()

# Mostrar resultados
print("Valores únicos por columna:\n")
print(conteo_unicos)
```

```
Valores únicos por columna:

provincia_id            23
provincia_nombre        23
departamento_id        129
departamento_nombre    413
id_cultivo              36
cultivo                 42
ciclo                   54
dtype: int64
```

4 Síntesis del EDA (Exploratory Data Analysis)

4.1 1. Revisión inicial

- Confirmar dimensiones del dataset (`df.shape`).
- Revisar tipos de datos (`df.dtypes`).
- Contar valores únicos en columnas clave (Provincia, Departamento, Cultivo, Ciclo).
- Obtener estadísticos básicos (`df.describe()`).

4.2 2. Calidad de datos

- Verificar duplicados (`df.duplicated().sum()`).
- Revisar valores nulos (`df.isnull().sum()`).
- Confirmar ausencia de inconsistencias lógicas (ej. `sup_sembrada = 0` con producción > 0).

4.3 3. Distribución de variables

- Histogramas de `sup_sembrada`, `sup_cosechada`, `produccion`, `rendimiento`.
- Conteo de cultivos (`cultivo.value_counts()`).
- Conteo de ciclos (`ciclo.value_counts()`).

4.4 4. Relaciones entre variables

- Scatter plots: `sup_sembrada vs produccion`, `sup_cosechada vs rendimiento`.
- Matriz de correlación (`sns.heatmap`).

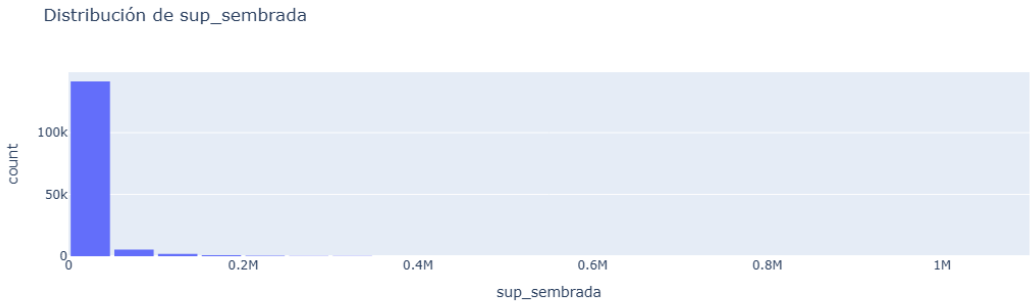
4.5 5. Análisis geográfico

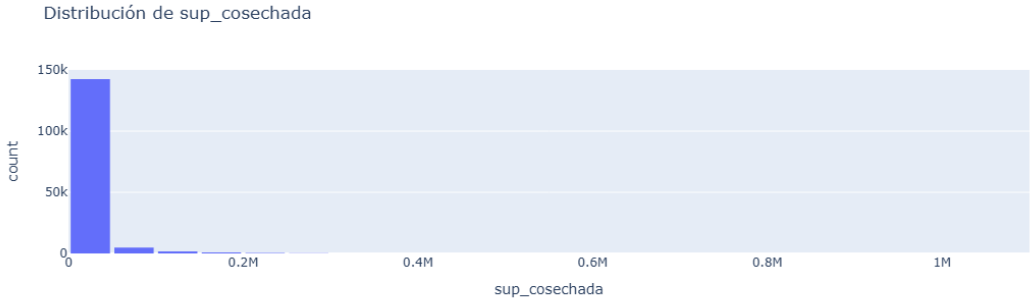
- Agrupar por `provincia_nombre` y `departamento_nombre`:
  - Superficie sembrada total.
  - Producción total.
  - Rendimiento promedio.
- Visualizaciones comparativas (gráficos de barras, mapas).

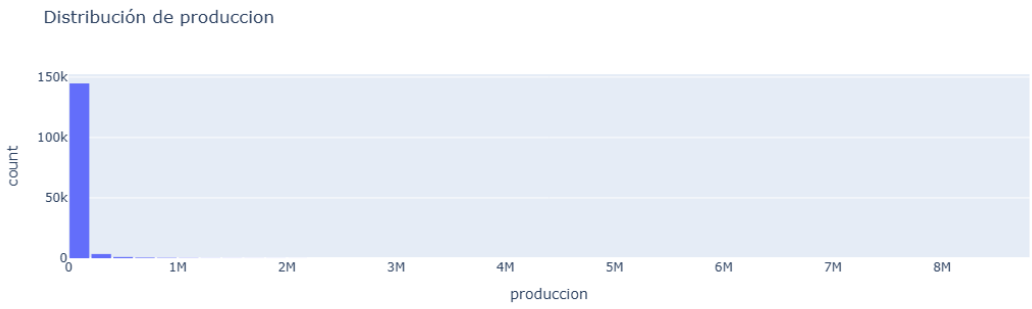
4.6 6. Detección de out

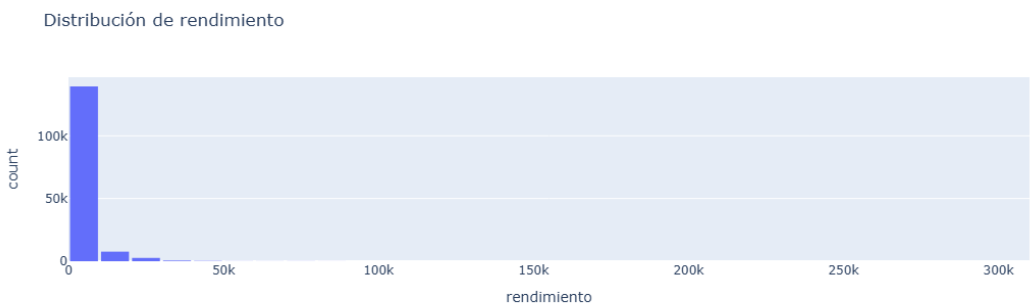
4.6.1 Histogramas globales de variables numéricas

[24]:
```
import plotly.express as px

num_vars = ["sup_sembrada", "sup_cosechada", "produccion", "rendimiento"]

for col in num_vars:
    fig = px.histogram(df, x=col, nbins=50, title=f"Distribución de {col}")
    fig.update_layout(bargap=0.1)
    fig.show()
```



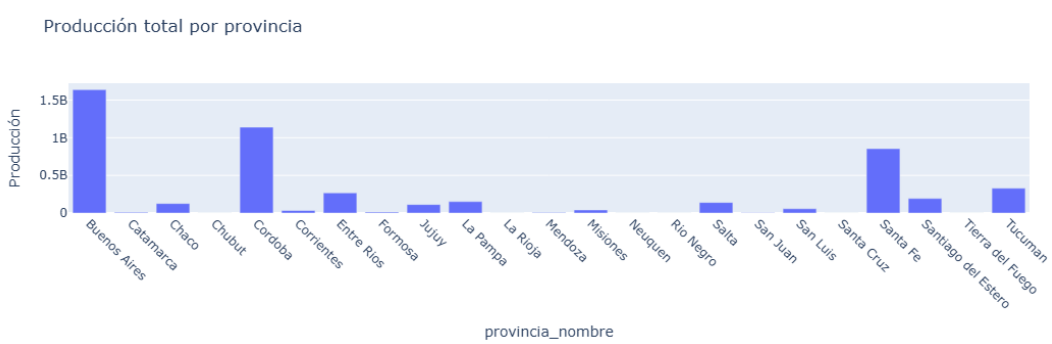






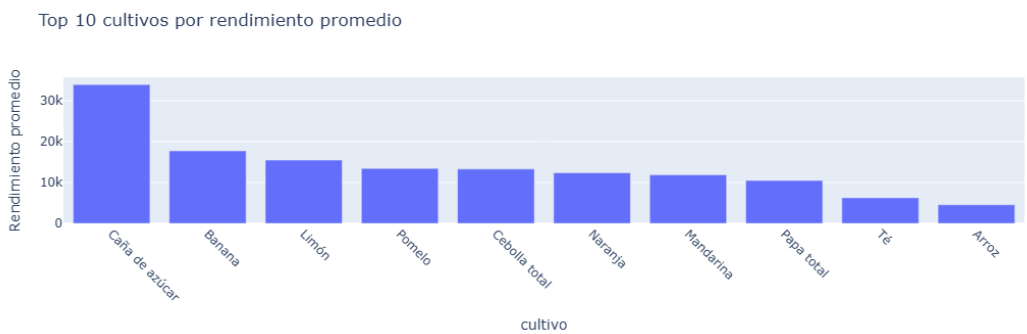
4.6.2 Producción total por provincia

[25]:
```
prod_prov = df.groupby("provincia_nombre")["produccion"].sum().reset_index()
fig = px.bar(prod_prov, x="provincia_nombre", y="produccion",
             title="Producción total por provincia",
             labels={"produccion":"Producción"})
fig.update_xaxes(tickangle=45)
fig.show()
```



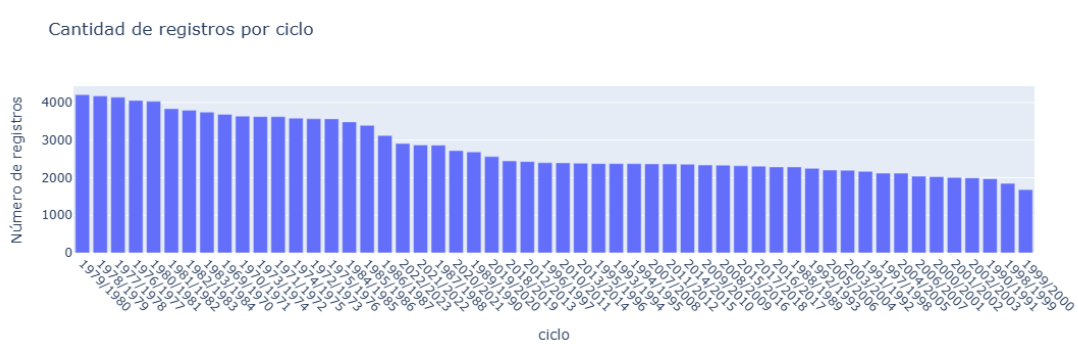
4.6.3 Rendimiento promedio por cultivo (Top 10)

[26]:
```
rend_cultivo = df.groupby("cultivo")["rendimiento"].mean().
 ↪sort_values(ascending=False).head(10).reset_index()
fig = px.bar(rend_cultivo, x="cultivo", y="rendimiento",
             title="Top 10 cultivos por rendimiento promedio",
             labels={"rendimiento":"Rendimiento promedio"})
fig.update_xaxes(tickangle=45)
fig.show()
```



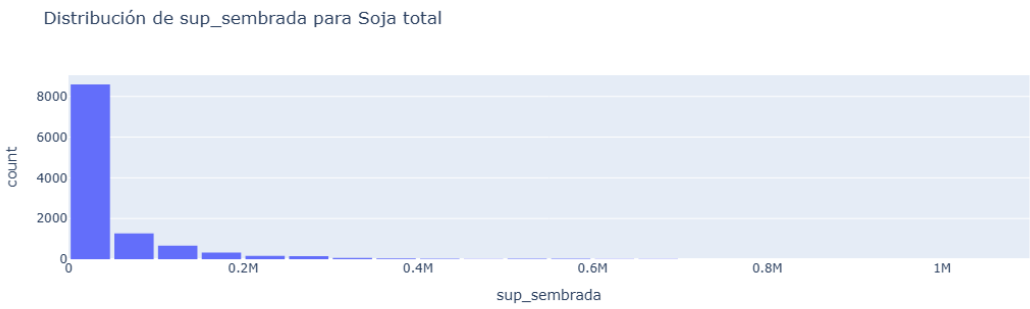
4.6.4 Conteo de registros por ciclo

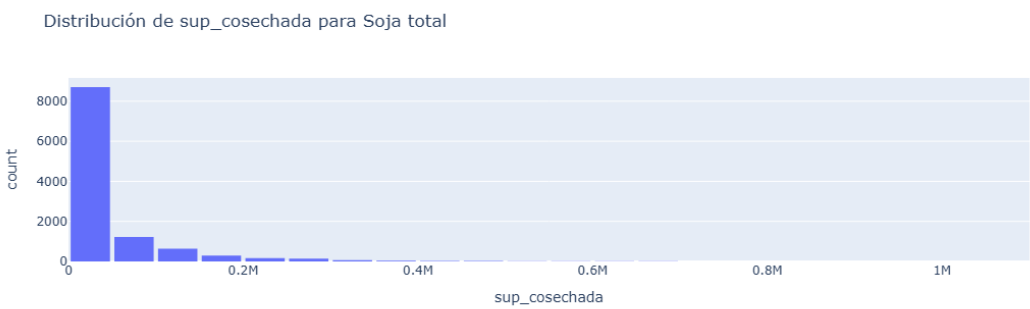
[27]:
```
ciclo_count = df["ciclo"].value_counts().reset_index()
ciclo_count.columns = ["ciclo", "conteo"]
fig = px.bar(ciclo_count, x="ciclo", y="conteo",
             title="Cantidad de registros por ciclo",
             labels={"conteo":"Número de registros"})
fig.update_xaxes(tickangle=45)
fig.show()
```

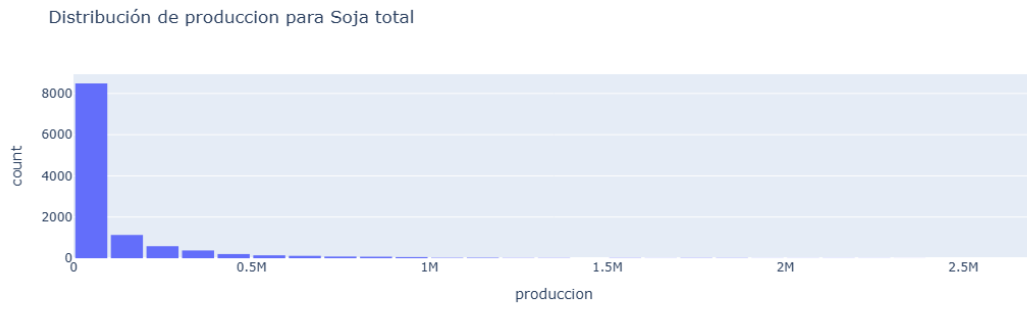


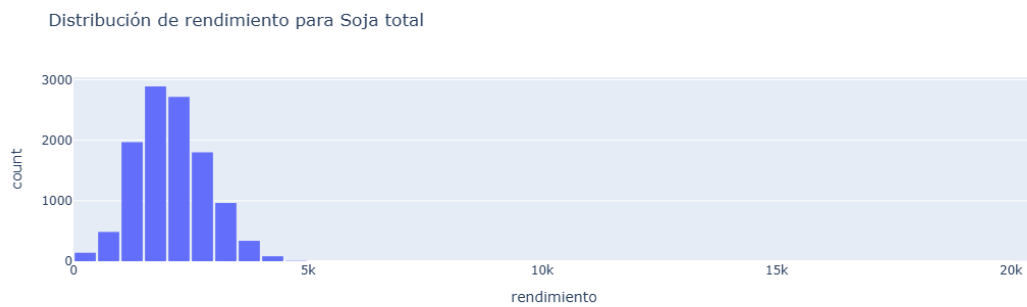
4.6.5 Cultivos mas importantes

[28]:
```
import plotly.express as px

# Filtrar registros de Soja total
df_soja_total = df[df["cultivo"].str.lower().str.contains("soja total")]

# Variables numéricas de interés
num_vars = ["sup_sembrada", "sup_cosechada", "produccion", "rendimiento"]

# Generar histogramas interactivos para cada variable
for col in num_vars:
    fig = px.histogram(df_soja_total, x=col, nbins=50,
                       title=f"Distribución de {col} para Soja total",
                       labels={col:col})
    fig.update_layout(bargap=0.1)
    fig.show()
```

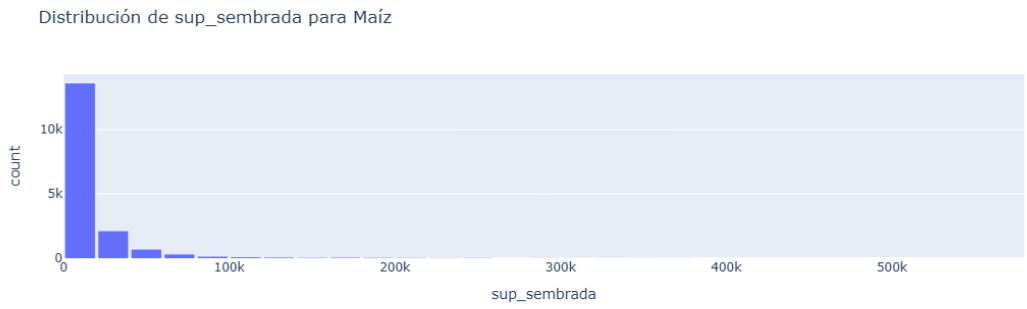


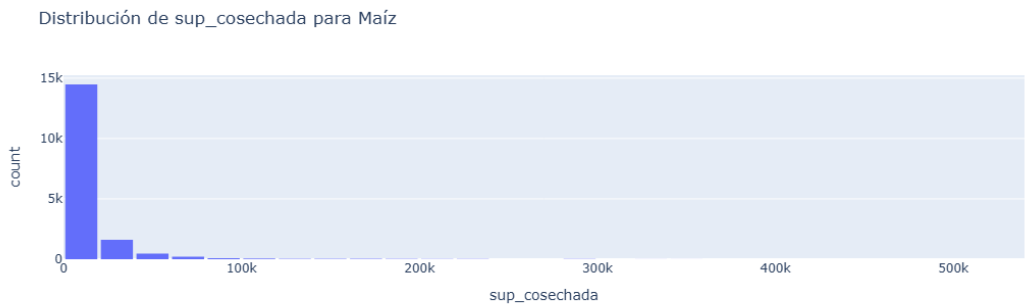


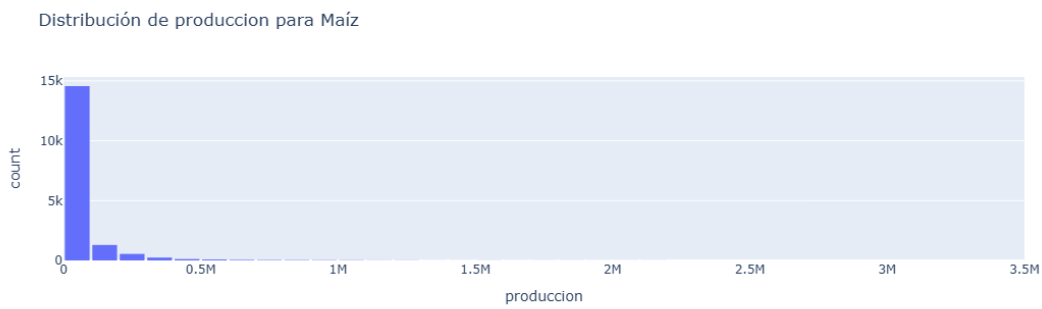


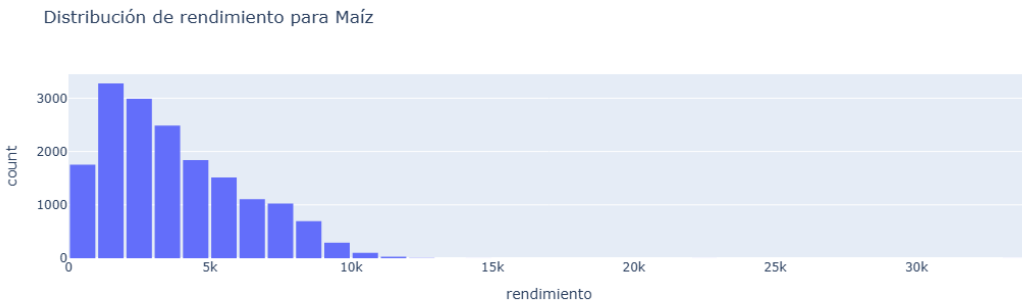


[29]:
```
import plotly.express as px

# Filtrar registros de Maíz
df_maiz = df[df["cultivo"].str.lower().str.contains("maíz")]

# Variables numéricas de interés
num_vars = ["sup_sembrada", "sup_cosechada", "produccion", "rendimiento"]

# Generar histogramas interactivos para cada variable
for col in num_vars:
    fig = px.histogram(df_maiz, x=col, nbins=50,
                       title=f"Distribución de {col} para Maíz",
                       labels={col:col})
    fig.update_layout(bargap=0.1)
    fig.show()
```


Distribución de sup_sembrada para Maíz
count
10k
5k
0
0
100k
200k
300k
400k
500k
sup_sembrada


Distribución de sup_cosechada para Maíz
count
15k
10k
5k
0
0
100k
200k
300k
400k
500k
sup_cosechada


Distribución de produccion para Maíz
count
15k
10k
5k
0
0
0.5M
1M
1.5M
2M
2.5M
3M
3.5M
produccion



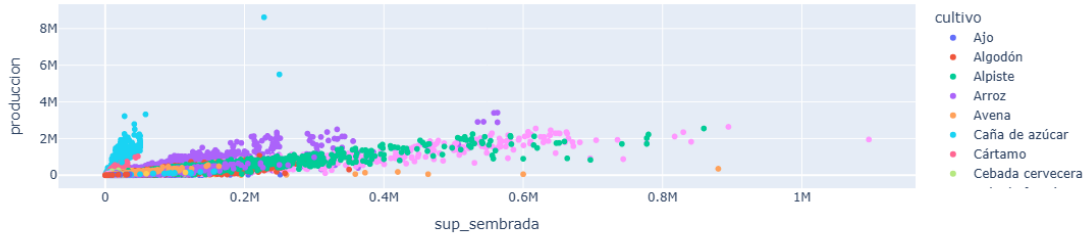
[ ]:

```

```

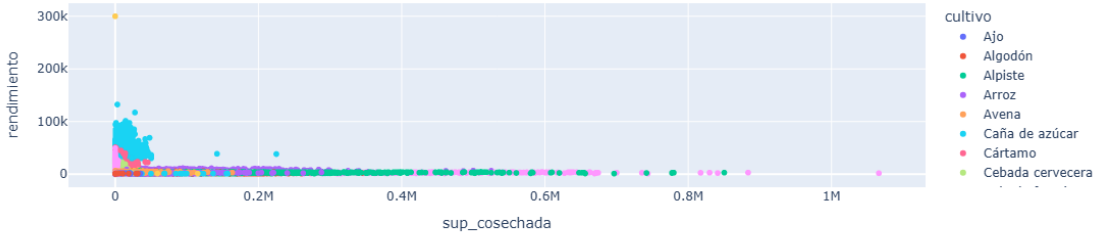
[30]:

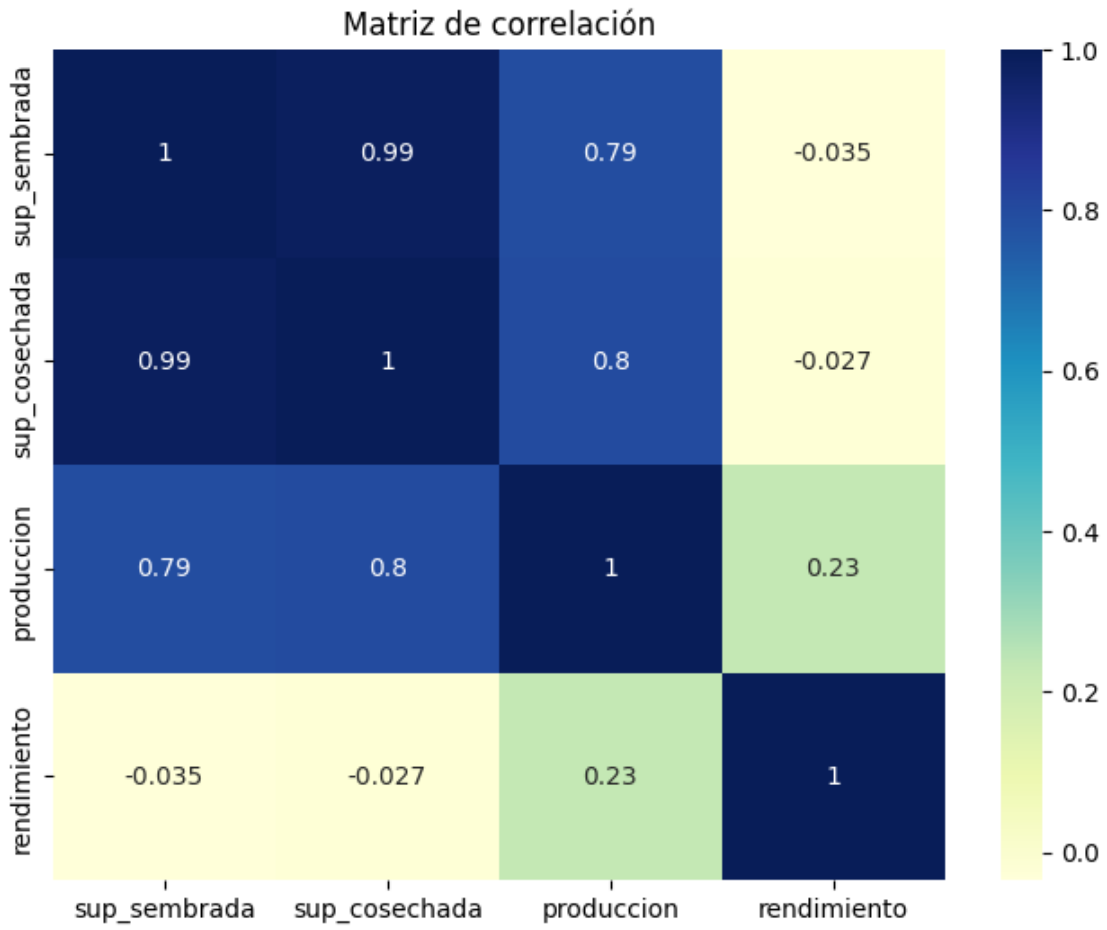
```
import plotly.express as px
import seaborn as sns
import matplotlib.pyplot as plt

# Scatter plot: superficie sembrada vs producción
fig = px.scatter(df, x="sup_sembrada", y="produccion",
                 color="cultivo",  # opcional: segmentar por cultivo
                 title="Relación entre superficie sembrada y producción")
fig.show()

# Scatter plot: superficie cosechada vs rendimiento
fig = px.scatter(df, x="sup_cosechada", y="rendimiento",
                 color="cultivo",  # opcional: segmentar por cultivo
                 title="Relación entre superficie cosechada y rendimiento")
fig.show()

# Matriz de correlación
plt.figure(figsize=(8,6))
sns.heatmap(df[["sup_sembrada","sup_cosechada","produccion","rendimiento"]].
 ↪corr(),
            annot=True, cmap="YlGnBu")
plt.title("Matriz de correlación")
plt.show()
```

Relación entre superficie sembrada y producción



Relación entre superficie cosechada y rendimiento



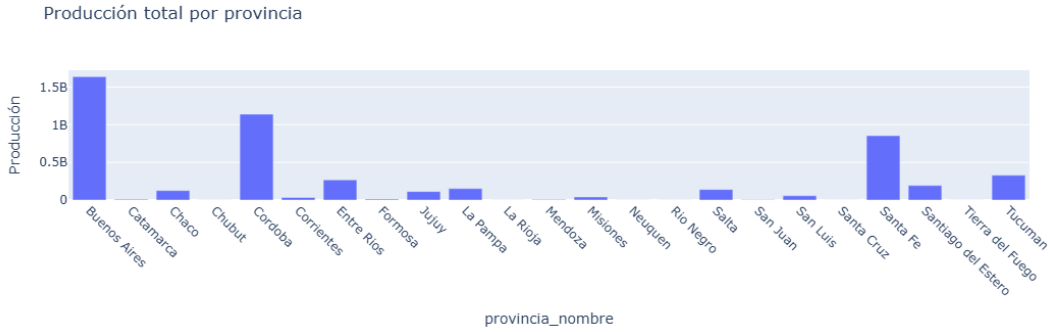


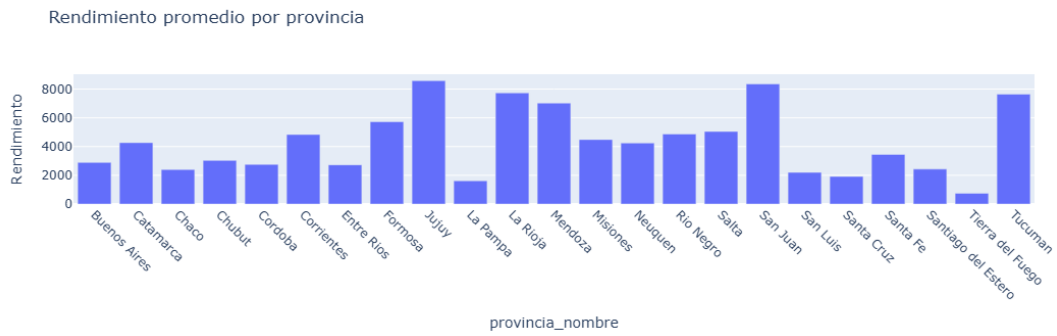
4.6.6 Análisis geográfico

Agrupación por provincia

[31]:
```
import plotly.express as px

# Agrupación por provincia
prov_stats = df.groupby("provincia_nombre").agg({
    "sup_sembrada":"sum",
    "produccion":"sum",
    "rendimiento":"mean"
}).reset_index()

# Superficie sembrada total por provincia
fig = px.bar(prov_stats, x="provincia_nombre", y="sup_sembrada",
             title="Superficie sembrada total por provincia",
             labels={"sup_sembrada":"Superficie sembrada"})
fig.update_xaxes(tickangle=45)
fig.show()
```

```python
# Producción total por provincia
fig = px.bar(prov_stats, x="provincia_nombre", y="produccion",
             title="Producción total por provincia",
             labels={"produccion":"Producción"})
fig.update_xaxes(tickangle=45)
fig.show()

# Rendimiento promedio por provincia
fig = px.bar(prov_stats, x="provincia_nombre", y="rendimiento",
             title="Rendimiento promedio por provincia",
             labels={"rendimiento":"Rendimiento"})
fig.update_xaxes(tickangle=45)
fig.show()
```





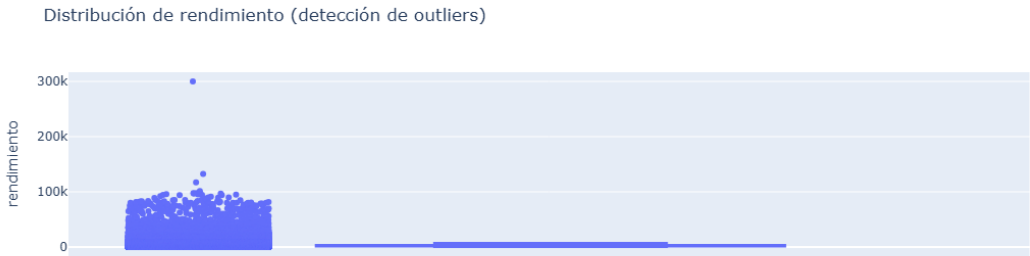
Agrupación por departamento

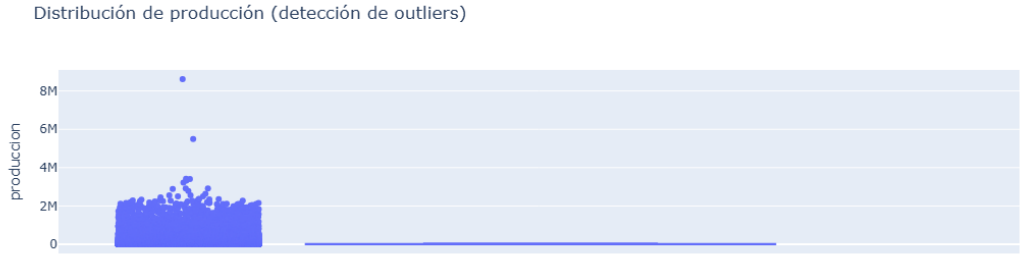
[32]:
```
# Agrupación por departamento
dept_stats = df.groupby("departamento_nombre").agg({
    "sup_sembrada":"sum",
    "produccion":"sum",
    "rendimiento":"mean"
}).reset_index()

# Top 15 departamentos por producción
top_dept_prod = dept_stats.sort_values("produccion", ascending=False).head(15)

fig = px.bar(top_dept_prod, x="departamento_nombre", y="produccion",
             title="Top 15 departamentos por producción",
             labels={"produccion":"Producción"})
fig.update_xaxes(tickangle=45)
fig.show()
```

4.6.7 Detección de outliers

Boxplots interactivos

[33]:
```
import plotly.express as px

# Boxplot de rendimiento
fig = px.box(df, y="rendimiento",
             title="Distribución de rendimiento (detección de outliers)",
             points="all")  # muestra todos los puntos
fig.show()

# Boxplot de producción
fig = px.box(df, y="produccion",
             title="Distribución de producción (detección de outliers)",
             points="all")
fig.show()
```

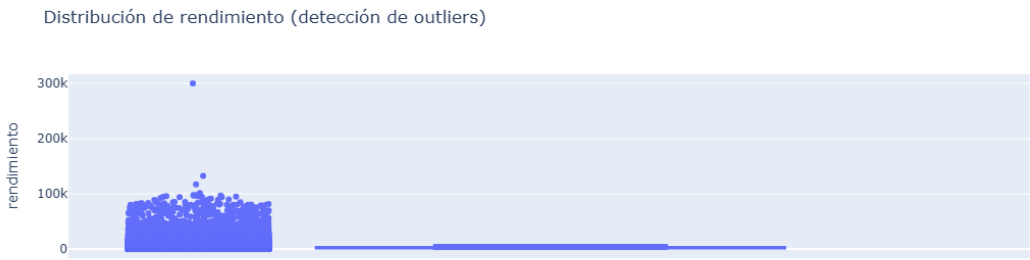


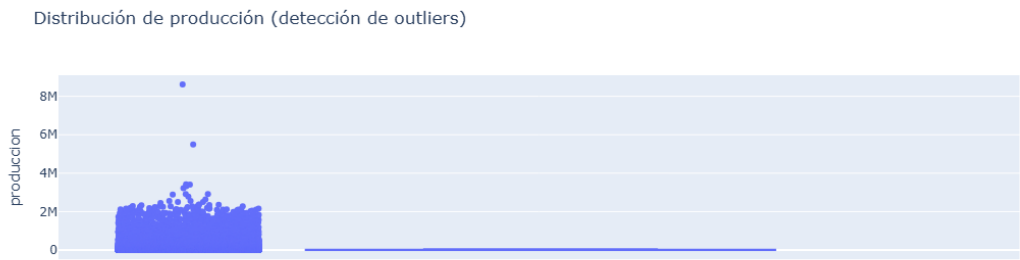


Percentiles extremos

[34]:
```
import plotly.express as px

# Boxplot de rendimiento
fig = px.box(df, y="rendimiento",
             title="Distribución de rendimiento (detección de outliers)",
```

```
             points="all")  # muestra todos los puntos
fig.show()

# Boxplot de producción
fig = px.box(df, y="produccion",
             title="Distribución de producción (detección de outliers)",
             points="all")
fig.show()
```





5 Modelo Estrella para Análisis Agrícola

El modelo estrella es una arquitectura de datos diseñada para optimizar el análisis en herramientas de BI como Power BI.
Se compone de una **tabla central de hechos** y varias **tablas de dimensiones** que rodean a la tabla de hechos, permitiendo segmentar y filtrar la información.

5.1 Tabla de Hechos: `fact_agricola`

Contiene las métricas cuantitativas y las claves foráneas que conectan con las dimensiones.

Campo	Descripción
provincia_id	Identificador único de la provincia
departamento_id	Identificador único del departamento
id_cultivo	Identificador único del cultivo
ciclo	Ciclo agrícola (ej. campaña, año)
sup_sembrada	Superficie sembrada (hectáreas)
sup_cosechada	Superficie cosechada (hectáreas)
produccion	Producción total (toneladas)
rendimiento	Rendimiento promedio (toneladas/hectárea)

5.2 Tablas de Dimensiones

5.2.1 1. `dim_provincia`

Campo	Descripción
provincia_id	Identificador de provincia
provincia_nombre	Nombre de la provincia

5.2.2 2. `dim_departamento`

Campo	Descripción
departamento_id	Identificador de departamento
departamento_nombre	Nombre del departamento
provincia_id	Relación jerárquica con provincia

5.2.3 3. `dim_cultivo`

Campo	Descripción
id_cultivo	Identificador de cultivo
cultivo	Nombre del cultivo (ej. Soja total, Maíz, Trigo)

5.2.4 4. `dim_ciclo`

Campo	Descripción
ciclo	Ciclo agrícola (ej. año, campaña)

5.3 Relaciones en Power BI

- `fact_agricola.provincia_id` → `dim_provincia.provincia_id`
- `fact_agricola.departamento_id` → `dim_departamento.departamento_id`
- `fact_agricola.id_cultivo` → `dim_cultivo.id_cultivo`
- `fact_agricola.ciclo` → `dim_ciclo.ciclo`

5.4 Ventajas del Modelo Estrella

- **Claridad**: separación entre métricas (hechos) y descripciones (dimensiones).
- **Eficiencia**: evita duplicación de nombres y reduce tamaño de la tabla de hechos.
- **Flexibilidad**: permite segmentar por provincia, departamento, cultivo o ciclo.
- **Escalabilidad**: se pueden agregar nuevas dimensiones sin alterar la estructura central.

5.5 Exportación a CSV

Cada tabla debe guardarse como archivo CSV en la carpeta `Repos_1`:

- `fact_agricola.csv`
- `dim_provincia.csv`
- `dim_departamento.csv`
- `dim_cultivo.csv`
- `dim_ciclo.csv`

Estos archivos se importan en Power BI y se conectan mediante las relaciones definidas.

---

[35]:
```
df.shape
```

[35]: (150687, 14)

[39]:
```
import pandas as pd
from pathlib import Path

# ========= Configuración =========
BASE = Path(r"C:\Users\victo\OneDrive\Data_Science\Repos_1")
```

```python
SRC = BASE / "estim-agric-limpio.csv"

FACT_OUT = BASE / "fact_agricola.csv"
PROV_OUT = BASE / "dim_provincia.csv"
DEP_OUT = BASE / "dim_departamento.csv"
CUL_OUT = BASE / "dim_cultivo.csv"
CIC_OUT = BASE / "dim_ciclo.csv"
TIME_OUT = BASE / "dim_tiempo.csv"

# Logs de calidad
QA_DIR = BASE / "qa_logs"
QA_DIR.mkdir(exist_ok=True)
LOG_DUP_IDS = QA_DIR / "departamento_id_conflictos.csv"
LOG_DUP_NAMES = QA_DIR / "departamento_nombre_conflictos.csv"
LOG_NULLS = QA_DIR / "departamento_nulls.csv"

# ========= Carga =========
df = pd.read_csv(SRC)

# ========= Limpieza base =========
# Normaliza strings (espacios, mayúsculas/minúsculas) para evitar falsos␣
↪duplicados
def normalize_str(s):
    if pd.isna(s):
        return s
    return str(s).strip()

for col in ["departamento_nombre", "provincia_nombre", "ciclo", "cultivo"]:
    if col in df.columns:
        df[col] = df[col].map(normalize_str)

# Elimina registros con departamento_id inválido (0 o nulos)
invalid_mask = (df["departamento_id"].isna()) | (df["departamento_id"] == 0)
df_valid = df.loc[~invalid_mask].copy()

# Guarda log de registros inválidos
df.loc[invalid_mask].to_csv(LOG_NULLS, index=False)

# ========= Auditoría de integridad departamento =========
# 1) IDs con múltiples nombres/provincias
dup_id = (
    df_valid.groupby("departamento_id")
    .agg(
        nombres=("departamento_nombre", lambda x: sorted(set(x.dropna()))),
        provincias=("provincia_id", lambda x: sorted(set(x.dropna())))
    )
    .reset_index()
```

```
)
conflict_id_mask = (dup_id["nombres"].str.len() > 1) | (dup_id["provincias"].
 ↪str.len() > 1)
dup_id.loc[conflict_id_mask].to_csv(LOG_DUP_IDS, index=False)

# 2) Nombres con múltiples IDs (posibles homónimos o errores)
dup_name = (
    df_valid.groupby("departamento_nombre")
    .agg(ids=("departamento_id", lambda x: sorted(set(x.dropna()))),
         provincias=("provincia_id", lambda x: sorted(set(x.dropna()))))
    .reset_index()
)
conflict_name_mask = dup_name["ids"].str.len() > 1
dup_name.loc[conflict_name_mask].to_csv(LOG_DUP_NAMES, index=False)

# ========= Resolución de conflictos =========
# Regla: para cada departamento_id, elegir la combinación (nombre,␣
 ↪provincia_id) más frecuente.
# Esto asegura unicidad de ID y coherencia con la provincia donde más aparece.

freq = (
    df_valid.groupby(["departamento_id", "departamento_nombre", "provincia_id"])
    .size()
    .reset_index(name="freq")
)

# Elegir la fila con mayor frecuencia por departamento_id
freq_sorted = freq.sort_values(["departamento_id", "freq",␣
 ↪"departamento_nombre"], ascending=[True, False, True])
canonical = freq_sorted.groupby("departamento_id").head(1).
 ↪reset_index(drop=True)

# ========= Construcción de dimensiones limpias =========
# Dim_departamento: únicamente el mapeo canónico (1:1)
dim_departamento = canonical[["departamento_id", "departamento_nombre",␣
 ↪"provincia_id"]].copy()

# Dim_provincia: deducida del DF original (IDs válidos), luego depurada por␣
 ↪unicidad de provincia_id
dim_provincia = (
    df_valid[["provincia_id", "provincia_nombre"]]
    .dropna(subset=["provincia_id"])
    .drop_duplicates()
)
```

```python
# Para garantizar 1:1 en provincia_id, elegir el nombre más frecuente si␣
↪hubiera inconsistencias
prov_freq = (
    df_valid.groupby(["provincia_id", "provincia_nombre"])
    .size()
    .reset_index(name="freq")
    .sort_values(["provincia_id", "freq", "provincia_nombre"], ascending=[True,␣
↪False, True])
)
dim_provincia = prov_freq.groupby("provincia_id").head(1)[["provincia_id",␣
↪"provincia_nombre"]]

# Dim_cultivo
dim_cultivo = df_valid[["id_cultivo", "cultivo"]].drop_duplicates()

# Dim_ciclo
dim_ciclo = df_valid[["id_ciclo", "ciclo"]].drop_duplicates()

# Dim_tiempo
dim_tiempo = df_valid[["periodo_fecha", "periodo_anio"]].drop_duplicates()

# ========= Tabla de hechos =========
fact_table = df_valid[[
    "provincia_id",
    "departamento_id",
    "id_cultivo",
    "id_ciclo",
    "periodo_fecha",
    "periodo_anio",
    "sup_sembrada",
    "sup_cosechada",
    "produccion",
    "rendimiento"
]].copy()

# Opcional: asegurar que los IDs de la tabla de hechos existan en las␣
↪dimensiones (integridad referencial)
fact_table = fact_table.merge(dim_departamento[["departamento_id"]],␣
↪on="departamento_id", how="inner")
fact_table = fact_table.merge(dim_provincia[["provincia_id"]],␣
↪on="provincia_id", how="inner")

# ========= Exportación =========
fact_table.to_csv(FACT_OUT, index=False)
dim_provincia.to_csv(PROV_OUT, index=False)
dim_departamento.to_csv(DEP_OUT, index=False)
```

```python
dim_cultivo.to_csv(CUL_OUT, index=False)
dim_ciclo.to_csv(CIC_OUT, index=False)
dim_tiempo.to_csv(TIME_OUT, index=False)

# ========= Resumen de calidad =========
summary = {
    "filas_originales": len(df),
    "filas_invalidas_depId": int(invalid_mask.sum()),
    "conflictos_por_id": int(conflict_id_mask.sum()),
    "conflictos_por_nombre": int(conflict_name_mask.sum()),
    "dim_departamento_filas": len(dim_departamento),
    "dim_provincia_filas": len(dim_provincia),
    "hechos_filas": len(fact_table)
}
print("QA Summary:", summary)
print(f"Logs guardados en: {QA_DIR}")
```

```
QA Summary: {'filas_originales': 150687, 'filas_invalidas_depId': 592,
'conflictos_por_id': 29, 'conflictos_por_nombre': 43, 'dim_departamento_filas':
128, 'dim_provincia_filas': 23, 'hechos_filas': 150095}
Logs guardados en: C:\Users\victo\OneDrive\Data_Science\Repos_1\qa_logs
```